

Reviewer Pack — Minimal Order of Geometric Quantization and Resolution Proof...

Entry de824b8ad86b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 11:42:32 UTC

Minimal Order of Geometric Quantization and Resolution Proof Width Inequality

Entry ID: de824b8ad86b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 11:42:32 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with m clauses, the minimal order of geometric quantization ($o_q(\varphi)$) is linearly related to its resolution proof width $w(\varphi)$, such that $o_q(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Arithmetic geometry provides a rich framework for studying algebraic varieties over finite fields, which can be used to encode the structure of CNFs. The minimal order of geometric quantization measures the complexity of these varieties and is expected to correlate with the resolution proof width due to their shared algebraic underpinnings.

Taxonomy category: Arithmetic_Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3cd292b3ac04bcc9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for a set of N random CNFs with clause counts from 100 to 1000, the correlation coefficient between $o_q(\varphi)$ and $w(\varphi)$ exceeds 0.9 AND the p-value from a linear regression model is less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	SAFE	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric quantization" AND "resolution proof complexity" - "order of geometric quantization" AND "resolution proof width" - "arithmetic geometry" AND "minimal order in resolution proofs"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, 2*m), -random.randint(1, 2*m)]
            cnf.append(clause)
        return cnf

    def resolution_width(cnf):
        queue = set()
        for lit in set(x for clause in cnf for x in clause):
            queue.add(frozenset([lit]))

        while len(queue) > 0:
            i, j = random.sample(list(queue), 2)
            if i.intersection(j):
                new_lit = -i.intersection(j).pop()
                if not any(new_lit == -x for x in queue):
                    queue.add(frozenset([new_lit]))
            else:
                return len(queue)
        return len(queue)

    def geometric_quantization_order(cnf):
        # Placeholder function to simulate the computation
        # This is a dummy implementation and should be replaced with actual arithmetic geometry logic
        return random.randint(1, 10) * len(cnf)
```

```

m_values = [100, 200, 300, 400, 500]
o_q_values = []
w_phi_values = []

for m in m_values:
    cnf = generate_cnf(m)
    o_q_values.append(geometric_quantization_order(cnf))
    w_phi_values.append(resolution_width(cnf))

if len(o_q_values) < 30 or len(w_phi_values) < 30:
    return {
        "metric_name": "o_q vs w_phi",
        "metric_value": None,
        "instances_tested": len(o_q_values),
        "n_max": max(m_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

n = len(o_q_values)
mean_o_q = sum(o_q_values) / n
mean_w_phi = sum(w_phi_values) / n

covariance = sum((o_q_values[i] - mean_o_q) * (w_phi_values[i] - mean_w_phi) for i in range(n))
variance_o_q = sum((o_q_values[i] - mean_o_q) ** 2 for i in range(n)) / n
variance_w_phi = sum((w_phi_values[i] - mean_w_phi) ** 2 for i in range(n)) / n

correlation_coefficient = covariance / (math.sqrt(variance_o_q) * math.sqrt(variance_w_phi))

return {
    "metric_name": "o_q vs w_phi",
    "metric_value": correlation_coefficient,
    "instances_tested": len(o_q_values),
    "n_max": max(m_values),
    "conjecture_holds": abs(correlation_coefficient) > 0.9 and p_value < 0.05,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) /

    support_count = sum(1 for r in results if r["conjecture_holds"])
    support_fraction = Fraction(support_count, len(results)).limit_denominator()

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= Fraction(80, 100):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    for r in results:
        if not r["conjecture_holds"]:
            print(f"RESULT: FALSIFIED counterexample=\"not_enough_instances\" first_failing_seed={r['seed']}")
            break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

hi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
TRIAL: {'metric_name': 'o_q vs w_phi', 'metric_value': None, 'instances_tested': 5, 'n_max': 500, 'conjecture_holds': False, 'counterexample': None, 'seed': 1}
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e015c495.py", line 109, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"not_enough_instances\" first_failing_seed={r['seed']}")
                                         ~~~~~^~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to ensure it can complete without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11319
2	preregistration	ollama_remote	glm4:latest	0	0	12085

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	8330
4	novelty	ollama_remote	glm4:latest	0	0	11765
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14218
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12325
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12777
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11928
9	judge	ollama_remote	glm4:latest	0	0	11798

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106545 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/de824b8ad86b.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/de824b8ad86b.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/de824b8ad86b.tar.gz* (if generated)